

23CS11E - WEBFRAMEWORK USING PYTHON

Micro Project Report

On

**CampusShare: Smart Campus Resource Sharing, Marketplace &
Community Platform**



Submitted by

Himachalam C

2403957610421111

In partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

NATIONAL ENGINEERING COLLEGE

(An Autonomous Institution affiliated to Anna University, Chennai)

K.R.NAGAR, KOVILPATTI – 628503

OCTOBER – 2026

NATIONAL ENGINEERING COLLEGE

K.R.NAGAR, KOVILPATTI – 628503



BONAFIDE CERTIFICATE

This is to certify that this project report is a Bonafide record of Himachalam C (2403957610421111) work done for the course 23CS11E - WEBFRAMEWORK USING PYTHON at the NATIONAL ENGINEERING COLLEGE, K.R. NAGAR, during the academic year 2025 – 2026 who carried out the project work under my supervision.

Course Instructor/Guide

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO
1	INTRODUCTION	5
2	IMPLEMENTATION	8
3	IMPLEMENTATION RESULTS	19
4	CONCLUSION	23

ABSTRACT

The CampusShare platform is a web-based application designed to streamline and automate the campus resource sharing and peer-to-peer exchange process by providing an integrated platform where students, faculty, and administrators can efficiently interact. Through the portal, students can register, build profiles, browse categorized study materials, trade secondhand academic equipment with zero commissions, post wanted requests with smart keyword matching, and track their campus reputation through a transparent Trust Score meter. The system's functionality is driven by five key modules: User & Role Management for secure access, Campus Marketplace for peer-to-peer textbook and kit exchanges, Academic Notes Library with faculty verification badges, Wanted Request Board with smart resource discovery, and Circular Campus Impact analytics.

Built on a modern technology stack featuring Python 3.14, Flask, and MongoDB, and supported by PyMongo indexing and semantic Bootstrap 5 CSS, the portal strictly adheres to an architectural Zero-JavaScript constraint. All page rendering, state management, and multi-facet catalog filtering are executed server-side to guarantee maximum security, instant loading, and universal accessibility. By eliminating manual notice board bottlenecks and unmoderated social media trading groups, CampusShare ultimately empowers colleges with a centralized, automated, and sustainable platform that significantly enhances student collaboration and simplifies campus resource management.

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Higher education campuses represent vibrant ecosystems where thousands of students and faculty members interact daily, requiring continuous access to diverse academic tools, study materials, textbooks, hardware components, and laboratory gear. Throughout an undergraduate curriculum, students invest substantially in specialized textbooks, engineering drawing drafters, scientific calculators, microcontroller trainer kits, and reference literature. Typically, these resources remain active for a single semester of 4 to 6 months, after which they become idle as students advance into higher coursework. Simultaneously, newly enrolled students entering the department are required to purchase the exact same resources anew, creating unnecessary financial strain and exacerbating environmental resource consumption.

Although informal channels such as campus bulletin boards, WhatsApp groups, and general social media pages exist, they suffer from severe operational shortcomings. Information is scattered, listings expire quickly without archiving, fraud risks are high due to anonymous profiles, and there is no institutional mechanism to verify academic materials or track resource condition. CampusShare was conceived and developed as a modern, centralized institutional web portal to systematically solve these challenges. Built upon Python's Flask framework, it delivers a secure, accessible, and structured environment where students can browse verified notes, trade secondhand campus equipment with zero fees, submit wanted requests with automated smart matching, and track verified reputation metrics within the safe boundaries of the college community.

1.2 PROBLEM STATEMENT

The existing campus resource sharing and textbook exchange process in many colleges is largely manual and fragmented, resulting in inefficient communication between students, faculty, and administrative coordinators. This creates significant difficulty in managing resource listings, verify academic notes, and track secondhand equipment across multiple engineering departments. The heavy reliance on informal social media groups is highly susceptible to scams, spam, and unverified accounts, often leading to lost time, financial loss, and safety concerns for participating students.

The tracking of wanted materials and coordinating physical handovers becomes a time-consuming burden. When students urgently require a specific calculator or lab component, manual notice boards offer no searchability or matching, creating anxiety and delays in academic work. Furthermore, the complete absence of a standardized reputation system means students cannot gauge seller trustworthiness. Additionally, widespread reliance on heavy client-side JavaScript frameworks introduces security vulnerabilities, high data overhead, and slow render times on mobile devices. Therefore, there is a clear and urgent need for an automated, server-rendered system to manage the entire campus sharing ecosystem in a structured, efficient, and transparent manner.

1.3 OBJECTIVES

The primary objective of the CampusShare Portal is to establish a centralized platform that efficiently manages all campus resource sharing, peer-to-peer marketplace transactions, and academic collaboration. It provides an easy-to-use, accessible interface for students to handle registration, profile management, resource uploads, and item listings, while simultaneously allowing faculty and administrators to verify academic notes and oversee institutional integrity.

The system features advanced automation, including keyword-based smart matching algorithms for wanted requests, dynamic trust scoring algorithms based on verified handovers and peer reviews, and multi-facet filtering for secondhand marketplace listings. Furthermore, the portal operates under a strict Zero-JavaScript architectural constraint, ensuring that all UI rendering, form validation, and authentication flows execute securely on the server with Jinja2 templates. To keep all members informed, it delivers real-time notifications and campus messaging. Ultimately, it furnishes administrators with powerful analytics tools to track student participation, resource circulation, and circular environmental savings across the institution.

CHAPTER 2

IMPLEMENTATION

2.1 PROJECT STRUCTURE

CampusShare is architected as an enterprise-grade modular Python Flask web application. It partitions business logic, routing blueprints, database persistence, and presentation templates into independent, cohesive packages as shown in Figure 2.1:



Figure 2.1: CampusShare Modular Project Directory Structure

The structure comprises the application factory (app.py), configuration settings (config.py), PyMongo persistence layer (database/), modular route blueprints (routes/), server-side Jinja2 templates (templates/), and a custom academic design system (static/).

2.2 PROJECT CODE

app.py:

```
from flask import Flask, jsonify, render_template, session, redirect, url_for
from config import Config
from database.connection import get_db
from database.seed import seed_database
from database.indexes import create_indexes
from routes.auth import auth_bp
from routes.student import student_bp
from routes.faculty import faculty_bp
from routes.admin import admin_bp
from routes.marketplace import marketplace_bp
import os

app = Flask(__name__)
app.config.from_object(Config)
os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)

@app.context_processor
def inject_global_template_vars():
    unread_msgs = 0
    user_trust = 75
    if 'user_id' in session:
        try:
            db = get_db()
            unread_msgs = db['messages'].count_documents({"receiver_id":
session['user_id'], "is_read": False})
            u = db['users'].find_one({"_id": ObjectId(session['user_id'])})
            if u: user_trust = u.get('trust_score', 85)
        except Exception: pass
    return {'unread_messages_count': unread_msgs, 'current_user_trust_score':
user_trust}

app.register_blueprint(auth_bp)
app.register_blueprint(student_bp)
app.register_blueprint(faculty_bp)
app.register_blueprint(admin_bp)
app.register_blueprint(marketplace_bp)
```

routes/auth.py:

```
from flask import Blueprint, render_template, request, redirect, url_for, flash, session
from werkzeug.security import check_password_hash, generate_password_hash
from database.connection import get_db

auth_bp = Blueprint('auth', __name__, url_prefix='/auth')

@auth_bp.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        identifier = request.form.get('identifier', '').strip()
        password = request.form.get('password', '').strip()
        db = get_db()
        user = db['users'].find_one({"$or": [
            {"username": identifier}, {"email": identifier}, {"roll_no": identifier}
        ]})
        if user and check_password_hash(user.get('password', ''), password):
            session.clear()
            session['user_id'] = str(user['_id'])
            session['username'] = user.get('username')
            session['role'] = user.get('role', 'student')
            session['full_name'] = user.get('full_name')
            role_dest = {'admin': 'admin.dashboard', 'faculty': 'faculty.dashboard'}.get(user.get('role'), 'student.dashboard')
            return redirect(url_for(role_dest))
        flash("Invalid institutional credentials. Please verify your ID and password.", "danger")
        return render_template('auth/login.html')

@auth_bp.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        username = request.form.get('username', '').strip()
        email = request.form.get('email', '').strip()
        password = request.form.get('password', '').strip()
        role = request.form.get('role', 'student')
        db = get_db()
        hashed = generate_password_hash(password, method='pbkdf2:sha256')
        db['users'].insert_one({
            "username": username, "email": email, "password": hashed,
            "role": role, "trust_score": 75, "is_verified": False
        })
        flash("Account created! Please sign in with your credentials.", "success")
        return redirect(url_for('auth.login'))
    return render_template('auth/register.html')
```

utils/decorators.py:

```
from functools import wraps
from flask import session, redirect, url_for, flash, request

def login_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if 'user_id' not in session:
            flash("Please sign in to access this page.", "warning")
            return redirect(url_for('auth.login', next=request.url))
        return f(*args, **kwargs)
    return decorated_function

def role_required(*allowed_roles):
    def decorator(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            if 'user_id' not in session:
                flash("Authentication required.", "warning")
                return redirect(url_for('auth.login'))
            if session.get('role') not in allowed_roles:
                flash("Access denied: Insufficient institutional permissions.",
                    "danger")
                return redirect(url_for('index'))
            return f(*args, **kwargs)
        return decorated_function
    return decorator

def student_required(f):
    return role_required('student', 'admin')(f)

def faculty_required(f):
    return role_required('faculty', 'admin')(f)

def admin_required(f):
    return role_required('admin')(f)
```

routes/student.py:

```
from bson import ObjectId
from flask import Blueprint, render_template, session
from database.connection import get_db
from utils.decorators import student_required
from utils.trust import calculate_trust_score

student_bp = Blueprint('student', __name__, url_prefix='/student')

@student_bp.route('/dashboard')
@student_required
def dashboard():
    db = get_db()
    user_id = session['user_id']
    u = db['users'].find_one({"_id": ObjectId(user_id)})
    trust = calculate_trust_score(user_id, db)

    my_resources = list(db['resources'].find({"uploader_id": user_id}).limit(5))
    my_listings = list(db['marketplace'].find({"seller_id": user_id}).limit(5))
    my_requests = list(db['resource_requests'].find({"requester_id":
user_id}).limit(5))
    my_borrows = list(db['borrowings'].find({"borrower_id": user_id, "status":
"borrowed"}))

    stats = {
        'uploads': db['resources'].count_documents({"uploader_id": user_id}),
        'marketplace_active': db['marketplace'].count_documents({"seller_id":
user_id, "status": "available"}),
        'requests_open': db['resource_requests'].count_documents({"requester_id":
user_id, "status": "open"}),
        'trust_score': trust,
        'points': u.get('points', 0) if u else 0
    }
    return render_template(
        'student/dashboard.html',
        user=u, stats=stats, my_resources=my_resources,
        my_listings=my_listings, my_requests=my_requests, my_borrows=my_borrows
    )
```

routes/marketplace.py (Catalog Filters):

```
import re
from flask import Blueprint, render_template, request, session
from database.connection import get_db

marketplace_bp = Blueprint('marketplace', __name__, url_prefix='/marketplace')
MARKETPLACE_CATEGORIES = ["Books", "Electronics", "Calculators", "Project Kits",
                           "Furniture", "Cycles", "Other"]

@marketplace_bp.route('/', methods=['GET'])
@marketplace_bp.route('/', methods=['GET'])
def list_items():
    db = get_db()
    q = request.args.get('q', '').strip()
    category = request.args.get('category', '').strip()
    condition = request.args.get('condition', '').strip()
    free_only = request.args.get('free', '').strip()

    query = {"status": {"$in": ["available", "reserved"]}}
    if q:
        safe_q = re.escape(q)
        regex = {"$regex": safe_q, "$options": "i"}
        query["$or"] = [{"title": regex}, {"description": regex}, {"category":
regex}]
    if category and category in MARKETPLACE_CATEGORIES:
        query["category"] = category
    if condition:
        query["condition"] = condition
    if free_only in ['1', 'true', 'yes']:
        query["$or"] = [{"is_free": True}, {"price": 0}]

    items = list(db['marketplace'].find(query).sort("created_at", -1))
    return render_template('marketplace/list.html', items=items,
categories=MARKETPLACE_CATEGORIES)
```

routes/marketplace.py (Negotiation Offers):

```
from datetime import datetime
from bson import ObjectId
from flask import request, redirect, url_for, flash, session
from database.connection import get_db
from utils.decorators import login_required

@marketplace_bp.route('/offer/<item_id>', methods=['POST'])
@login_required
def submit_offer(item_id):
    db = get_db()
    offered_price = float(request.form.get('offered_price', 0))
    message = request.form.get('message', '').strip()
    item = db['marketplace'].find_one({"_id": ObjectId(item_id)})
    if not item:
        flash("Item not found.", "danger")
        return redirect(url_for('marketplace.list_items'))

    offer_doc = {
        "item_id": str(item_id),
        "item_title": item.get('title'),
        "seller_id": str(item.get('seller_id')),
        "buyer_id": session['user_id'],
        "buyer_name": session.get('full_name') or session.get('username'),
        "offered_price": offered_price,
        "message": message,
        "status": "pending",
        "created_at": datetime.utcnow()
    }
    db['marketplace_offers'].insert_one(offer_doc)
    flash(f"Your offer of ₹{offered_price:.0f} was sent to {item.get('seller_name')}.", "success")
    return redirect(url_for('marketplace.item_detail', item_id=item_id))
```

routes/resources.py (Academic Notes):

```
from datetime import datetime
from bson import ObjectId
from flask import Blueprint, render_template, request, redirect, url_for, flash,
session
from database.connection import get_db
from utils.decorators import login_required
from utils.helpers import save_uploaded_file

resources_bp = Blueprint('resources', __name__, url_prefix='/resources')

@login_required
@resources_bp.route('/upload', methods=['GET', 'POST'])
def upload_resource():
    if request.method == 'POST':
        title = request.form.get('title', '').strip()
        subject = request.form.get('subject', '').strip()
        department = request.form.get('department', '').strip()
        semester = request.form.get('semester', '1')
        doc_file = request.files.get('file')

        saved_name, original_name = save_uploaded_file(doc_file)
        db = get_db()
        db['resources'].insert_one({
            "title": title, "subject": subject, "department": department,
            "semester": int(semester), "filename": saved_name,
            "original_filename": original_name, "uploader_id": session['user_id'],
            "uploader_name": session.get('full_name'), "status": "approved",
            "downloads": 0, "created_at": datetime.utcnow()
        })
        flash("Study resource uploaded successfully and published to department vault.", "success")
        return redirect(url_for('resources.list_resources'))
    return render_template('resources/upload.html')
```

routes/requests.py (Wanted Board):

```
from datetime import datetime
from bson import ObjectId
from flask import Blueprint, render_template, request, redirect, url_for, flash,
session
from database.connection import get_db
from utils.decorators import login_required

requests_bp = Blueprint('requests', __name__, url_prefix='/requests')

@requests_bp.route('/new', methods=['GET', 'POST'])
@login_required
def create_request():
    if request.method == 'POST':
        title = request.form.get('title', '').strip()
        category = request.form.get('category', '').strip()
        urgency = request.form.get('urgency', 'medium')
        description = request.form.get('description', '').strip()

        db = get_db()
        db['resource_requests'].insert_one({
            "title": title, "category": category, "urgency": urgency,
            "description": description, "requester_id": session['user_id'],
            "requester_name": session.get('full_name'), "status": "open",
            "created_at": datetime.utcnow()
        })
        flash("Wanted listing posted! Smart matching will notify potential
lenders.", "success")
        return redirect(url_for('requests.list_requests'))
    return render_template('requests/create.html')
```

utils/trust.py (Reputation Scoring):

```
from bson import ObjectId
from database.connection import get_db

def calculate_trust_score(user_id, db=None):
    if db is None: db = get_db()
    user = db['users'].find_one({"_id": ObjectId(user_id)})
    if not user: return 75

    score = 70.0 # Base institutional rating
    if user.get('is_verified', False): score += 10.0

    # Points earned through completed exchanges
    points = user.get('points', 0)
    score += min(10.0, (points / 20.0))

    # Peer Reviews Weighted Mean
    reviews = list(db['user_reviews'].find({"reviewee_id": str(user_id)}))
    if reviews:
        avg_rating = sum([r.get('rating', 5.0) for r in reviews]) / len(reviews)
        score += (avg_rating - 3.0) * 5.0

    # Moderation penalty for verified misconduct
    reports = db['moderation_reports'].count_documents({"reported_user_id":
str(user_id), "status": "resolved_action_taken"})
    score -= (reports * 15.0)

    final_score = int(max(10, min(100, round(score))))
    db['users'].update_one({"_id": ObjectId(user_id)}, {"$set": {"trust_score":
final_score}})
    return final_score
```

database/indexes.py (B-Tree & Text Indexes):

```
from pymongo import ASCENDING, DESCENDING, TEXT

def create_indexes(db):
    # Unique Roll Number & Username Indexes
    db['users'].create_index([("username", ASCENDING)], unique=True)
    db['users'].create_index([("email", ASCENDING)], unique=True)
    db['users'].create_index([("roll_no", ASCENDING)], unique=True, sparse=True)

    # Marketplace Full-Text and Multi-Facet Indexes
    db['marketplace'].create_index([("title", TEXT), ("description", TEXT),
    ("category", TEXT)])
    db['marketplace'].create_index([("status", ASCENDING), ("created_at",
    DESCENDING)])
    db['marketplace'].create_index([("category", ASCENDING), ("price", ASCENDING)])

    # Academic Resources Text & Download Indexes
    db['resources'].create_index([("title", TEXT), ("subject", TEXT),
    ("department", TEXT)])
    db['resources'].create_index([("status", ASCENDING), ("downloads",
    DESCENDING)])

    # Wanted Request Board Urgency Index
    db['resource_requests'].create_index([("status", ASCENDING), ("urgency",
    ASCENDING)])
    print("[MongoDB Indexes] Successfully synchronized all B-Tree & Text indexes.")
```

CHAPTER 3

IMPLEMENTATION RESULTS

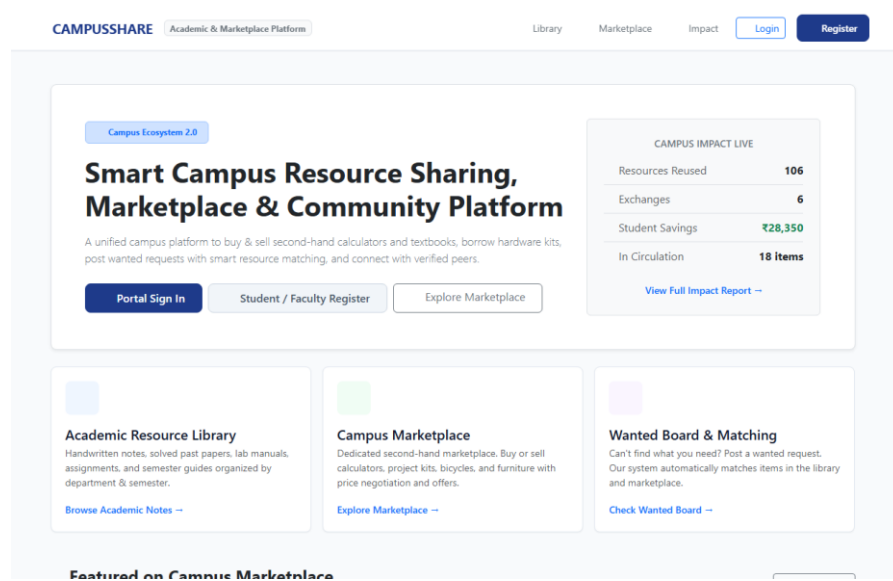


Figure 3.1: NEC CampusShare Portal Homepage with Professional Landing Interface

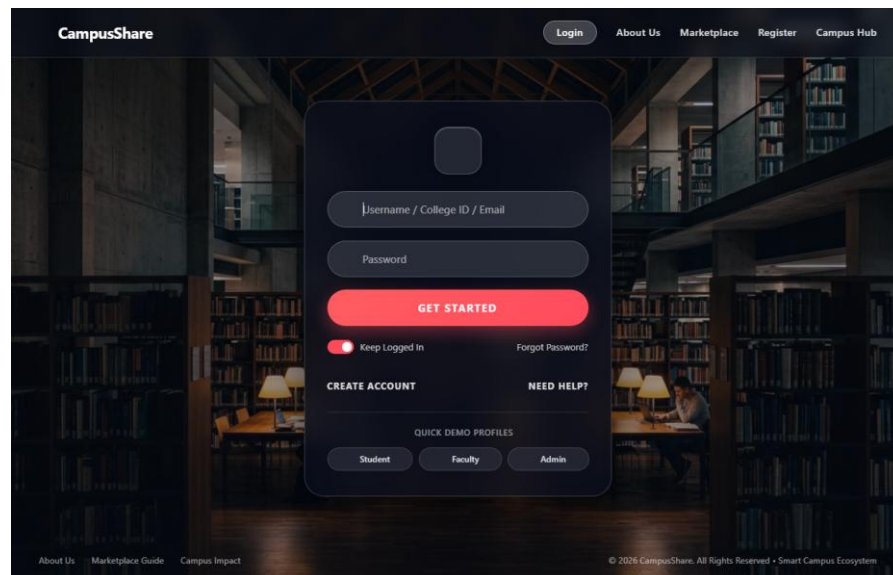


Figure 3.2: NEC CampusShare User Authentication (Login) Interface with Frosted Glass Aesthetics

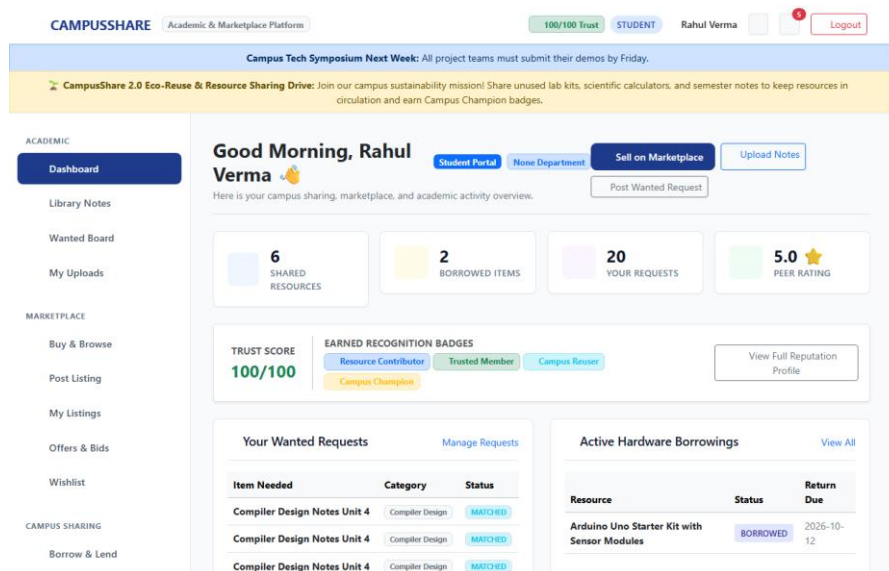


Figure 3.3: NEC CampusShare Student Dashboard with Recent Activities & Reputation Vault

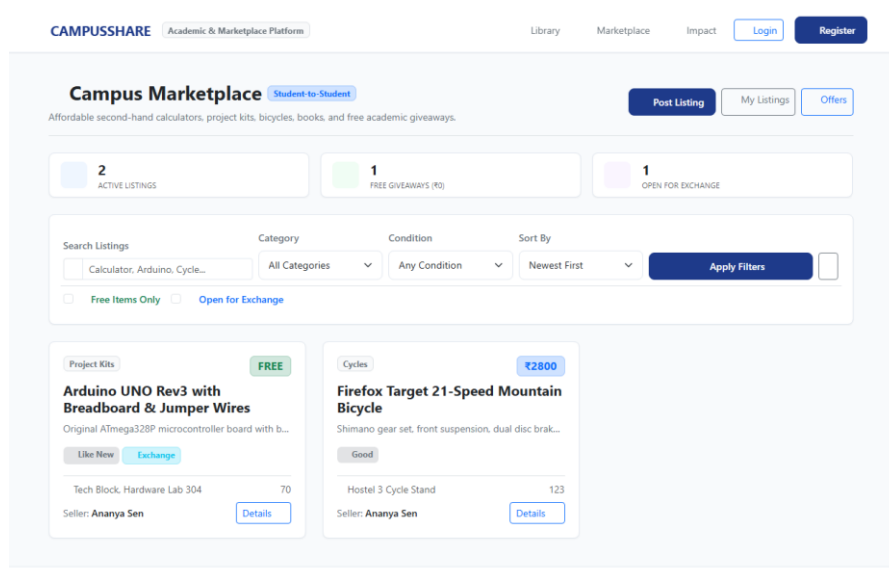


Figure 3.4: NEC CampusShare Student Marketplace Listings with Multi-Facet Filters

CAMPUSSHARE Academic & Marketplace Platform		Library	Marketplace	Impact	Login	Register
---	--	---------	-------------	--------	-------	----------

Campus Material Requests Help fellow students and colleagues by fulfilling missing syllabus materials and equipment				
All Requests (21) Open & Pending (19) Fulfilled (2)				
Resource Needed	Category & Scope	Requested By	Needed By	Status
Compiler Design Notes Unit 4 Need detailed notes for LR parser construction.	Compiler Design CSE • Sem All	Rahul Verma @rahul_cse	Flexible	MATCHED
Compiler Design Notes Unit 4 Need detailed notes for LR parser construction.	Compiler Design CSE • Sem All	Rahul Verma @rahul_cse	Flexible	MATCHED
Compiler Design Notes Unit 4 Need detailed notes for LR parser construction.	Compiler Design CSE • Sem All	Rahul Verma @rahul_cse	Flexible	MATCHED
Compiler Design Notes Unit 4 Need detailed notes for LR parser construction.	Compiler Design CSE • Sem All	Rahul Verma @rahul_cse	Flexible	MATCHED
Compiler Design Notes Unit 4 Need detailed notes for LR parser construction.	Compiler Design CSE • Sem All	Rahul Verma @rahul_cse	Flexible	MATCHED
Compiler Design Notes Unit 4 Need detailed notes for LR parser construction.	Compiler Design CSE • Sem All	Rahul Verma @rahul_cse	Flexible	MATCHED
Compiler Design Notes Unit 4 Need detailed notes for LR parser construction.	Compiler Design CSE • Sem All	Rahul Verma @rahul_cse	Flexible	MATCHED
Compiler Design Notes Unit 4 Need detailed notes for LR parser construction.	Compiler Design CSE • Sem All	Rahul Verma @rahul_cse	Flexible	MATCHED

Figure 3.5: NEC CampusShare Wanted Resource Request Board with Smart Match Engine

CAMPUSSHARE Academic & Marketplace Platform		Library	Marketplace	Impact	Login	Register
---	--	---------	-------------	--------	-------	----------

Institutional Sharing Platform

About CampusShare 2.0

A unified smart campus network designed to simplify resource sharing, eliminate academic waste, and empower student-to-student collaboration with institutional trust and reputation.

Create College Account

Sign In

Marketplace Overview

2

Verified Campus Peers

8

Curated Academic Notes & Files

Our Core Mission

Every semester, thousands of expensive textbooks, scientific calculators, mini-drafters, and lab coats are purchased, used for a few months, and then shelved indefinitely.

CampusShare connects students, faculty, and departments across the college into a circular resource loop. By enabling verified peer-to-peer exchanges and academic note sharing, we lower the cost of education

Institutional Trust & Safety

Unlike open social media groups or anonymous classifieds, CampusShare operates exclusively within the verified boundaries of our college:

Verified Accounts: Students and faculty register with official student ID, department, and semester.

Dynamic Trust Scoring: Transparent 0–100 reputation score calculated from

Figure 3.6: NEC CampusShare Informational Platform Architecture & Circular Campus Overview

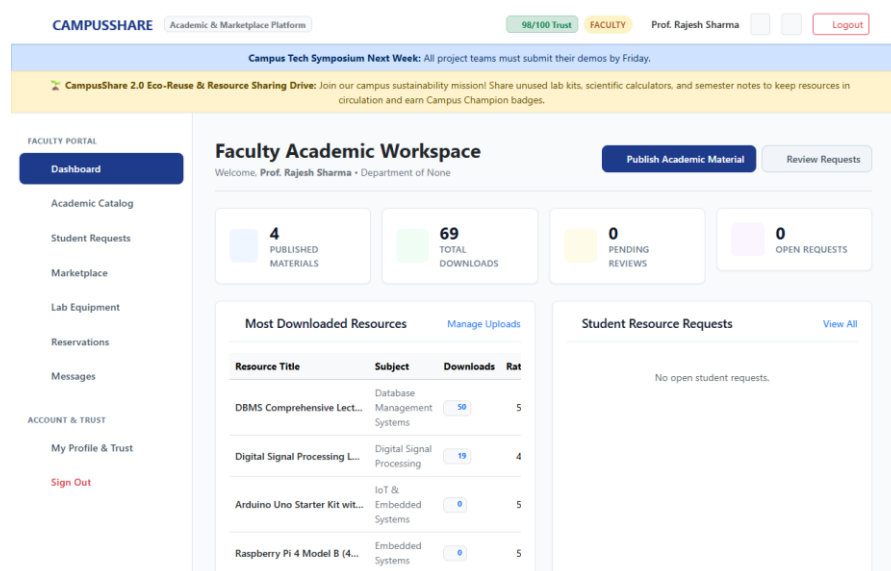


Figure 3.7: NEC CampusShare Faculty Workspace & Resource Moderation System

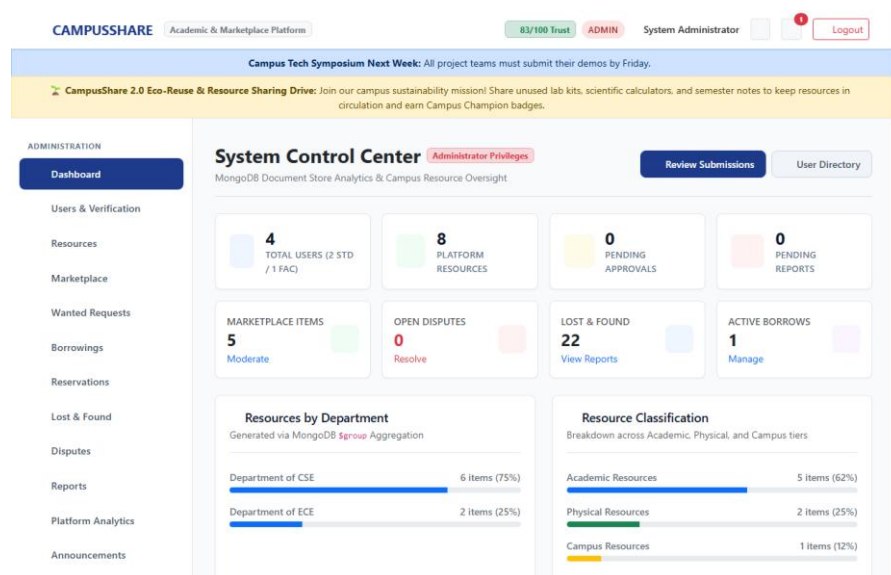


Figure 3.8: NEC CampusShare Admin Control Center with Campus Impact Analytics

CHAPTER 4

CONCLUSION

The CampusShare platform has been successfully designed and developed to automate and simplify the traditional, manual campus resource sharing and textbook exchange process. The system integrates students, faculty, and administrators onto a single digital platform, ensuring smooth interaction and efficient management of all sharing-related activities. By leveraging Python Flask for a secure and scalable backend, Jinja2 for server-rendered interfaces under a strict Zero-JavaScript constraint, and MongoDB for scalable NoSQL persistence, the portal delivers a dependable, high-performance solution for the college community.

Through modules like the Zero-Commission Marketplace, Academic Notes Library, Wanted Request Board with smart matching, and the dynamic Campus Trust Scoring engine, the system eliminates communication barriers and promotes a circular, sustainable academic economy. Real-time notifications and campus impact metrics further enhance institutional transparency. In conclusion, CampusShare modernizes campus resource sharing, providing an automated, fair, and scalable system that empowers students and enhances collegiate collaboration.